

CCE1 REVISED EXAMINATION QUESTION BANK — MASTER MODEL SOLUTIONS

Course Pattern: TE IT - 2024 Pattern (SPPU)

Academic Term: Semester V (2025-26 / 2026-27)

Document Type: Master In-Depth Model Solutions (Revised QB)

Course Code: PEC-304-IT / PEC-321A-IT

IMPORTANT:

Consolidated Question Bank Architecture (Revised): This authoritative document synthesizes and groups all 24 questions from the official revised CCE1 Question Bank (Unit 1: 10 Questions, Unit 2: 14 Questions) into **11 Comprehensive Topic Solutions**. Every section explicitly maps the original Question Bank IDs, Bloom's Taxonomy Levels (BTL), and marks weightage. Formatted with point-wise breakdowns, high-contrast parameter comparison tables, and labeled vector diagrams to guarantee maximum scoring in SPPU theory examinations.

Unit 1: Introduction to Cloud Computing, Services & Deployment Models (CO302.1)

Topic 1.1: Cloud Computing Foundations, Evolution, Need, Definition, Importance, Advantages & Limitations

GROUP:

Consolidated Questions Covered:

- **Q1:** Explain the need and evolution of Cloud Computing from traditional computing to cloud-based computing. [BTL2]
- **Q2:** Define Cloud Computing. Explain its basic concept and importance in modern computing. [BTL2]
- **Q3:** Describe the major advantages and limitations of Cloud Computing. [BTL2]

1. Academic & NIST Statutory Definition of Cloud Computing: According to the **National Institute of Standards and Technology (NIST SP 800-145)**:

*"Cloud computing is a model for enabling ubiquitous, convenient, **on-demand network access** to a **shared pool of configurable computing resources** (e.g., networks, servers, storage, applications, and services) that can be **rapidly provisioned and released** with **minimal management effort** or service provider interaction."*

2. Basic Concept & Importance in Modern Computing:

- **Resource Abstraction & Utility Model:** Cloud computing transforms computing power from an expensive capital asset into a metered utility service (analogous to electricity or water grids). Users consume compute, storage, and software over the internet without owning physical hardware.
- **Importance in Modern Computing Ecosystems:**
 1. **Foundation for Modern Digital Economy:** Powers global SaaS applications, mobile backends, streaming services (Netflix, Spotify), and enterprise ERPs.
 2. **Enabler for AI, ML & Big Data Analytics:** Provides instantaneous access to thousands of high-performance GPU/TPU clusters required for training LLMs and processing multi-petabyte datasets that no single startup could afford on-premises.

3. **Business Agility & Rapid Time-to-Market:** Developers can provision complete staging, testing, and production microservice environments in seconds via infrastructure-as-code (IaC) APIs.

3. Need & Economic Drivers (CapEx to OpEx Shift):

- **Capital Expenditure (CapEx) to Operational Expenditure (OpEx):** Eliminates massive upfront capital investments in datacenter real estate, server racks, diesel backup generators, and HVAC cooling. Organizations pay only for actual compute hours and gigabytes consumed on a pay-as-you-go basis.
- **Elimination of Idle Over-Provisioning:** Traditional datacenters are permanently sized for theoretical peak traffic (resulting in an abysmal 10% – 15% average utilization). The cloud dynamically auto-scales to match real-time workload fluctuations.

4. Chronological Evolution Roadmap from Traditional Computing:

1. **Mainframe Computing (1950s–1960s):** Centralized powerful mainframes (IBM System/360) accessed via dumb cathode-ray terminals; time-sharing enabled multiple users to run batch jobs concurrently.
2. **Client-Server & Distributed Computing (1980s–1990s):** Decentralized computing where desktop clients communicate with dedicated back-end database and application servers over local area networks (LANs).
3. **Cluster & Grid Computing (1990s–2000s):** Aggregating geographically distributed, heterogeneous computers across wide area networks to solve massive computational problems in science and engineering.
4. **Utility Computing:** Early commercial attempts to package compute cycles and storage as a metered utility service.
5. **Virtualization & Cloud-Native Platforms (2006–Present):** Hypervisors and container engines abstract physical silicon into multi-tenant, elastic pools delivered over broadband internet (pioneered by AWS EC2/S3 in 2006).

5. Parameter-Based Comparison: Traditional On-Premises vs. Modern Cloud Computing:

Parameter	Traditional On-Premises Computing	Modern Cloud Computing
Cost Model	Heavy upfront Capital Expenditure (CapEx) for hardware, real estate, cooling, plus perpetual software licenses.	Zero upfront CapEx; 100% Operational Expenditure (OpEx) with pay-as-you-go utility billing.
Scalability & Elasticity	Vertical scaling only; purchasing and installing physical servers takes weeks or months; hard hardware limits.	Elastic horizontal and vertical scaling in seconds via automated APIs; near-infinite on-demand capacity.
Resource Utilization	Very low (10% – 20%) due to permanent over-provisioning for theoretical peak loads.	High average utilization (> 80%) enabled by multi-tenant virtualization and dynamic resource pooling.
Maintenance & Upgrades	Heavy burden on in-house IT staff for physical server repairs, cabling, OS patching, and power maintenance.	Maintenance fully abstracted and managed by cloud vendor; automated zero-downtime rolling updates.
Disaster Recovery (DR)	Expensive secondary datacenter build-out; manual, complex failover procedures.	Built-in multi-region geographic replication, automated failover, and guaranteed 99.99% + availability SLAs.
Deployment Speed	Slow provisioning cycle (procurement, delivery, racking, OS installation takes 30 – 90 days).	Near-instantaneous provisioning (1 – 5 minutes) via Web Console, CLI, or Terraform scripts.

6. Major Advantages & Critical Limitations of Cloud Computing:

- **Major Advantages:**
 1. **Cost Efficiency:** Eliminates maintenance staff overhead, bulk discounts, and pay-per-second billing.
 2. **Global Reach & Low Latency:** Deploy applications worldwide across dozens of edge regions in minutes.
 3. **Elastic Scalability & High Availability:** Automatic horizontal pod/instance scaling with multi-AZ failover.
 4. **Continuous Innovation:** Immediate access to managed AI models, serverless runtimes, and distributed databases without manual installation.
- **Critical Limitations:**
 1. **Internet & Network Dependency:** Outages in ISP networks or cloud backbones cut off access to business data.

2. **Vendor Lock-In:** Proprietary cloud APIs (e.g., AWS DynamoDB, BigQuery) make migrating to another cloud vendor costly and technically challenging.
3. **Security, Privacy & Compliance Risks:** Storing sensitive customer data in multi-tenant public environments requires compliance with strict data sovereignty laws (GDPR, HIPAA, RBI localization rules).
4. **Unpredictable Operational Costs:** Unmonitored cloud resources, forgotten test instances, and runaway API calls can cause bill shock.

Source: Buyya et al., Ch 1, p. 3-19 | Source: Srinivasan & Suresh, Ch 1, p. 1-28

Topic 1.2: Cloud Service Models (IaaS, PaaS, SaaS) & Startup Application Scenario

GROUP:

Consolidated Questions Covered:

- **Q4:** Explain the IaaS, PaaS, and SaaS service models with suitable examples. [BTL2]
- **Q5:** Compare IaaS, PaaS, and SaaS based on user responsibility, infrastructure management, and applications. [BTL3]
- **Q9:** A startup plans to use cloud services for developing, deploying, and hosting its applications. Illustrate how IaaS, PaaS, and SaaS can be used in this scenario. Compare these service models based on user responsibility, infrastructure management, and applications. [BTL3]

1. Architectural Breakdown of Core Cloud Service Models: Cloud computing delivery models are organized into a three-tiered pyramid representing increasing levels of abstraction:

1. Infrastructure as a Service (IaaS):

- **Concept:** The cloud vendor provides raw, fundamental computing resources over the network: virtual machines, physical bare-metal servers, raw block storage, virtual private clouds (VPCs), and software-defined firewalls.
- **User Control:** The customer has root/administrative access to install and configure operating systems, middle-ware, database engines, runtimes, and applications.
- **Representative Examples:** Amazon Web Services (AWS EC2, EBS, VPC), Microsoft Azure VMs, Google Compute Engine (GCE), DigitalOcean Droplets.

2. Platform as a Service (PaaS):

- **Concept:** The cloud vendor delivers a fully managed application development and deployment runtime environment. Hardware, operating system, database engines, web servers, and container orchestrators are managed by the provider.
- **User Control:** Developers focus entirely on writing application source code and managing database schemas; they have no access to the underlying OS or hardware.
- **Representative Examples:** AWS Elastic Beanstalk, Google App Engine, Heroku, Microsoft Azure App Services, Red Hat OpenShift.

3. Software as a Service (SaaS):

- **Concept:** The cloud vendor hosts, manages, and delivers complete, ready-to-use software applications accessible to end users over web browsers or mobile apps.
- **User Control:** End users have zero management responsibility over infrastructure or application code; they configure user access, settings, and business workflows.
- **Representative Examples:** Google Workspace (Gmail, Drive, Docs), Microsoft 365, Salesforce CRM, Slack, Zoom, Dropbox.

2. Parameter-Based Comparison Matrix (Shared Responsibility Model):

Comparison Parameter	IaaS	PaaS	SaaS
Target Users	System Architects, Cloud Engineers, DevOps & Infrastructure Teams.	Software Developers, Web Engineers, Database Programmers, QA Testers.	End Users, Business Executives, Sales Teams, General Consumers.

Comparison Parameter	IaaS	PaaS	SaaS
User Responsibilities	Applications, Data, Runtime, Middleware, Operating System, Security Patches.	Application Source Code, Data Schemas, Configuration Parameters only.	User Accounts, Access Rights, Data Entry, Application Preferences only.
Provider Responsibilities	Physical Datacenter, Power, HVAC, Physical Servers, Hypervisor VMM, Networking.	Physical Hardware, Hypervisor, Host OS, Runtime, Middleware, Auto-scaling.	100% of Infrastructure, OS, Code, Databases, Backups, Updates, Security.
Level of Control	Highest; full root/admin control over virtual operating systems and networking.	Moderate; control over application logic and database queries; zero OS access.	Lowest; configuration and consumer feature utilization only.
Cost Model	Metered per vCPU-hour, RAM GB, Disk IOPS, and egress data transfer.	Metered per compute instance runtime, API invocations, and memory allocation.	Subscription per user seat / month or tiered consumption pricing.
Suitable Applications	Custom kernel tuning, legacy software migrations, deep learning training.	Rapid web/mobile app development, REST APIs, microservices, prototyping.	Standard business productivity: Email, CRM, ERP, Video Conferencing, Billing.

3. Startup Application Scenario Analysis:

- **Scenario:** A high-growth FinTech startup is building a digital payments and lending application from scratch. To maximize developer velocity and minimize capital expenditure, the startup strategizes its cloud adoption across all three service models:

1. SaaS Layer (Operational Productivity):

- The startup adopts **Google Workspace** for business email, **Slack** for team communication, **GitHub** for source code version control, and **Jira** for agile sprint tracking. This enables day-1 business productivity with zero infrastructure provisioning.

2. PaaS Layer (Core Web & Mobile API Backend):

- The startup deploys its customer-facing Node.js / Python REST API backend and mobile endpoints on **AWS Elastic Beanstalk** / **Google App Engine**, backed by managed **AWS RDS PostgreSQL**. The engineering team focuses purely on payment business logic, while the PaaS handles automated OS security patching, SSL termination, load balancing, and auto-scaling during traffic spikes.

3. IaaS Layer (Proprietary AI Credit-Scoring Engine & Custom Databases):

- For its proprietary fraud detection algorithms and AI creditworthiness neural networks, the startup provisions **AWS EC2 GPU instances (G4dn)** attached to **EBS Block Storage**. This provides low-level CUDA hardware acceleration and dedicated networking necessary for sub-millisecond fraud scoring.

Source: Srinivasan & Suresh, Ch 2, p. 35-55 | Source: Buyya et al., Ch 1, p. 8-15

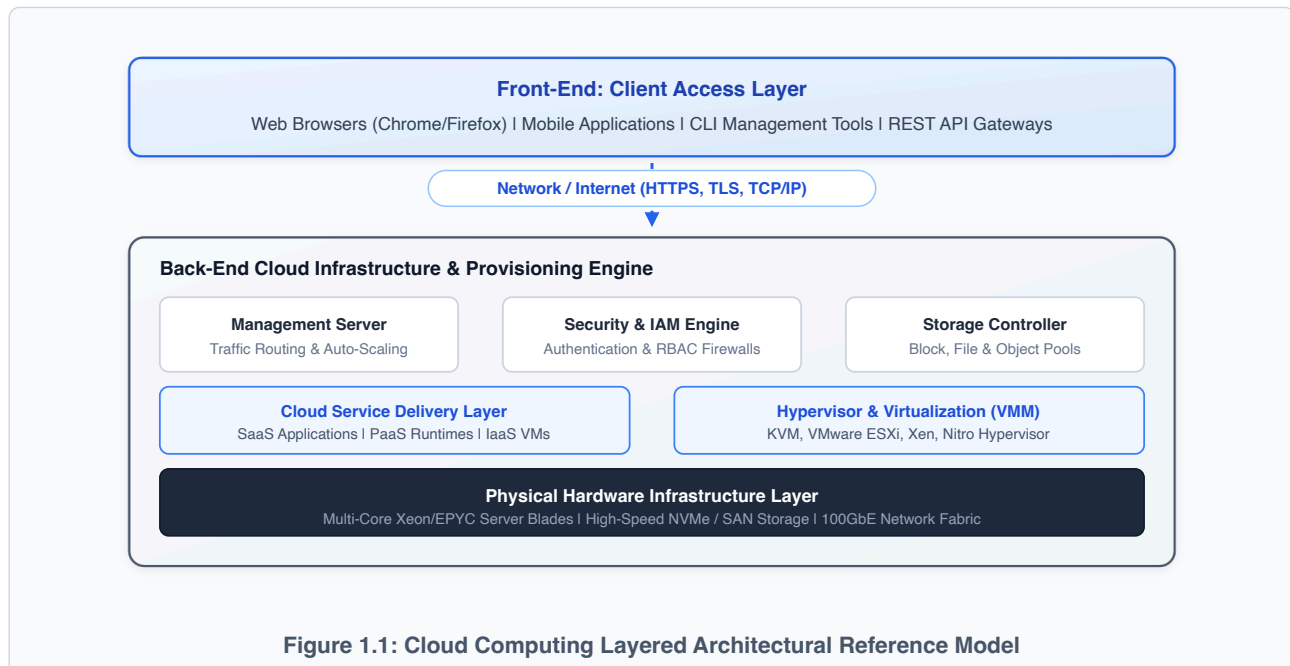
Topic 1.3: Cloud Computing Layered Architecture & Enterprise Application Evaluation

GROUP:

Consolidated Questions Covered:

- **Q6:** Illustrate the major components of Cloud Architecture with a suitable architectural diagram. [BTL3]
- **Q7:** Construct and explain the architecture of Cloud Computing. Analyze its major advantages and limitations with respect to enterprise applications. [BTL4]

1. Cloud Computing Layered Reference Architecture: Cloud architecture is organized into two primary segments connected across secure network backbones: the **Front-End Layer** (client-side user interfaces and endpoints) and the **Back-End Layer** (cloud provider datacenter resources, management software, and physical infrastructure).



2. Detailed Breakdown of Major Architectural Components:

- 1. Front-End (Client Layer):** Includes client devices (laptops, mobile smartphones, IoT devices), web browsers (Chrome, Firefox), thin clients, CLI management utilities, and API gateways that provide users and applications with access to cloud services via secure protocols (HTTPS, TLS, SSH).
- 2. Management & Orchestration Server:** The brain of the cloud datacenter. It coordinates traffic flow, dynamically provisions virtual resources, executes auto-scaling policies, performs workload balancing, and continuously monitors server node health.
- 3. Security & Identity Management (IAM) Engine:** Enforces multi-factor authentication (MFA), role-based access control (RBAC), data encryption (AES-256 in transit and at rest), and virtual firewall inspection.
- 4. Storage Controller Subsystem:** Manages distributed storage SANs and storage clusters, exposing block, file, and object storage abstractions with automated multi-zone replication.
- 5. Cloud Service Delivery Layer:** Implements the core service runtimes: SaaS applications, PaaS containers and database runtimes, and IaaS virtual machines.
- 6. Hypervisor / Virtual Machine Monitor (VMM):** The core virtualization abstraction layer (KVM, VMware ESXi, Xen) that decouples physical CPU, RAM, and PCIe hardware into isolated, multi-tenant virtual machines.
- 7. Physical Infrastructure Layer:** The physical datacenter hardware comprising high-density rack servers, high-speed fiber optical network switches, power distribution units, and storage disk arrays.

3. Enterprise Application Evaluation: Advantages & Limitations:

• Enterprise Advantages:

- **High Business Continuity & Disaster Recovery:** Multi-region active-active architectures eliminate single points of failure, ensuring 99.999% uptime for critical banking and ERP systems.
- **Global Scalability:** Easily absorbs sudden traffic surges during flash sales or annual financial reporting without purchasing permanent hardware.
- **Accelerated Digital Transformation:** Replaces monolithic legacy architectures with agile cloud-native microservices.

• Enterprise Limitations:

- **Compliance & Regulatory Hurdles:** Stringent industry mandates (PCI-DSS, HIPAA, GDPR) restrict hosting sensitive health or financial data in multi-tenant public environments.
- **Legacy Architectural Incompatibility:** Monolithic enterprise applications built on legacy mainframes or specific operating system kernels cannot be migrated to the cloud without costly refactoring.

- **Egress Bandwidth Costs:** High recurring data transfer fees when moving terabytes of operational data out of the cloud provider's network.

Source: Sosinsky, Ch 3, p. 45-68 | Source: Buyya et al., Ch 1 & 2, p. 20-45

Topic 1.4: Cloud Storage Concepts, Architecture & Significance

GROUP:

Consolidated Questions Covered:

- **Q8:** Explain Cloud Storage and describe its importance in cloud-based applications. [BTL2]

1. Academic Definition & Core Principles of Cloud Storage: **Cloud Storage** is a networked storage model where digital data is stored in logical pools across multiple physical storage servers owned and managed by a cloud hosting provider. The underlying physical environment spans multiple datacenters and storage arrays, presenting users with a virtually infinite, highly available, and durable storage endpoint accessed via standard network protocols and REST APIs.

Core Architectural Tenets:

- **Multi-Tenancy & Resource Pooling:** Physical disk arrays are dynamically partitioned and shared securely among millions of tenants.
- **High Durability & Availability:** Cloud storage providers implement automated data striping and erasure coding across multiple geographic availability zones, delivering up to 99.999999999% (11 nines) of annual data durability.
- **Pay-as-you-Go Metering:** Storage billing is calculated purely on gigabyte-months consumed, I/O operations performed, and data transferred.

2. Taxonomy of Cloud Storage Models:

Parameter	Block Storage	File Storage	Object Storage
Data Organization	Data split into raw, fixed-size blocks with unique sector addresses; no metadata.	Hierarchical tree directory structure of folders and files with standard file attributes.	Flat address space; data stored as immutable objects with unique IDs and custom metadata.
Access Protocol	Low-level SAN protocols: iSCSI, Fibre Channel, NVMe-oF.	Network file sharing protocols: NFS (Linux), SMB/CIFS (Windows).	Web RESTful APIs over HTTP/HTTPS (GET, PUT, DELETE requests).
Performance & Latency	Ultra-low sub-millisecond latency; highest IOPS.	Moderate latency; concurrent multi-client read/write access.	Higher latency; optimized for massive throughput and petabyte scaling.
Primary Use Cases	VM root boot drives, high-performance transactional relational databases (Oracle, MySQL).	Shared corporate file shares, content management systems (CMS), media editing pools.	Unstructured data: Big data lakes, static website hosting, long-term backups, media streaming.
Representative Services	AWS EBS, Azure Managed Disks, Google Persistent Disk.	AWS EFS, Azure Files, Google Cloud Filestore.	AWS S3, Azure Blob Storage, Google Cloud Storage.

3. Significance in Modern Cloud-Based Applications:

- 1. Decoupling Compute from Storage:** Modern cloud-native architectures separate compute nodes from storage nodes. Virtual machines or container pods can be terminated or scaled independently without risking data loss.
- 2. Enabler for Big Data Analytics & AI/ML Data Lakes:** Provides petabyte-scale repositories where raw telemetry, logs, and video datasets are stored cost-effectively before being processed by distributed engines like Apache Spark or Hadoop.

3. **Disaster Recovery & Automated Snapshotting:** Automated point-in-time volume snapshots enable near-instantaneous recovery from ransomware attacks or accidental deletions.

Source: Buyya et al., Ch 8, p. 215-240 | Source: Sosinsky, Ch 5, p. 110-135

Topic 1.5: Cloud Deployment Models & Multi-Department Organizational Recommendation Scenario

GROUP:

Consolidated Questions Covered:

- **Q10:** A large organization wants to adopt cloud computing for different departments. The finance department requires a dedicated and secure environment, the HR department needs to share resources with other organizations, and the development team requires flexible cloud resources. Illustrate how Public, Private, Hybrid, and Community Cloud deployment models can be considered for these requirements. Explain their characteristics, advantages, and suitable examples. [BTL3]

1. Detailed Architectural Analysis of Cloud Deployment Models:

1. Public Cloud:

- **Concept:** Cloud infrastructure is owned and operated by a third-party cloud provider (AWS, Microsoft, Google) and made available to the general public or industry enterprises over the internet.
- **Characteristics:** Multi-tenant shared physical infrastructure; near-infinite elasticity; zero customer CapEx; metered pay-as-you-go pricing.
- **Advantages:** Lowest total cost of ownership; zero hardware maintenance; rapid provisioning.
- **Examples:** Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP).

2. Private Cloud (Internal / Corporate Cloud):

- **Concept:** Cloud infrastructure is provisioned for exclusive use by a single organization comprising multiple business units. It may be owned, managed, and operated by the organization, a third party, or a combination of both, located on-premises or off-premises.
- **Characteristics:** Single-tenant isolation; dedicated hardware firewalls; maximum data privacy and control; customizable governance.
- **Advantages:** Absolute compliance with statutory banking/defense regulations; dedicated predictable performance; zero noisy-neighbor interference.
- **Examples:** On-premises OpenStack clusters, VMware vSphere Private Cloud, Microsoft Azure Stack.

3. Community Cloud:

- **Concept:** Cloud infrastructure is shared exclusively by several organizations that have shared concerns, missions, security requirements, policies, and compliance considerations (e.g., government departments, universities, banking consortiums).
- **Characteristics:** Multi-organization shared tenancy; collaborative governance; pooled operational costs.
- **Advantages:** Shared development and compliance costs; secure inter-agency data sharing; higher cost efficiency than dedicated private clouds.
- **Examples:** US GovCloud, Indian Banking Community Cloud (IDRBT), healthcare research clouds.

4. Hybrid Cloud:

- **Concept:** A composition of two or more distinct cloud infrastructures (Private, Community, or Public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load balancing).
- **Characteristics:** Interconnected via dedicated encrypted VPN or direct fiber connections (AWS Direct Connect / Azure ExpressRoute); unified orchestration plane.
- **Advantages:** Ultimate flexibility; keeps sensitive core data in private clouds while bursting non-sensitive workloads into public clouds during peak traffic.
- **Examples:** AWS Outposts integrated with AWS Public Cloud, Azure Hybrid Cloud.

2. Parameter-Based Deployment Comparison Matrix:

Parameter	Public Cloud	Private Cloud	Community Cloud	Hybrid Cloud
Infrastructure Ownership	Third-party Cloud Vendor.	Single Enterprise / Hosted.	Shared by Consortium.	Dual: Enterprise + Cloud Vendor.
Tenancy Model	Multi-tenant shared pool.	Strictly Single-tenant.	Multi-organization shared.	Hybrid (Single + Multi-tenant).
Security & Control	Standard / Shared model.	Highest / Full control.	High / Joint security policy.	Segmented by sensitivity.
Scalability	Near-infinite on-demand.	Limited by physical datacenter.	Limited by consortium budget.	Dynamic (Cloud Bursting).
Cost Structure	100% OpEx (Lowest cost).	High CapEx + Ongoing OpEx.	Shared pooled CapEx/OpEx.	Balanced CapEx and OpEx.
Representative Examples	AWS, Azure, GCP.	OpenStack, VMware Cloud.	GovCloud, BFSI Cloud.	AWS Outposts + AWS Public.

3. Multi-Department Enterprise Scenario Solution: Based on the specific organizational requirements, the deployment models are mapped as follows:

1. Finance Department → Private Cloud Recommendation:

- **Justification:** Financial transactions, employee payroll, and tax audits require dedicated single-tenant infrastructure to eliminate “noisy-neighbor” interference, prevent data exfiltration, and ensure compliance with strict financial regulatory frameworks (e.g., PCI-DSS, SOX, RBI guidelines).

2. HR Department → Community Cloud Recommendation:

- **Justification:** The HR department needs to collaborate and exchange employment verifications, shared recruitment talent pools, and industry-standard benefits data with partner organizations. A Community Cloud provides a secure, shared platform governed by unified inter-organizational compliance policies.

3. Development & QA Team → Public Cloud Recommendation:

- **Justification:** Software developers require highly flexible, self-service infrastructure to quickly spin up, test, and tear down experimental testbeds, staging containers, and CI/CD pipelines. Public Cloud provides instant elasticity and cost savings (paying only for the few hours test clusters run).

4. Enterprise Integration via Hybrid Cloud:

- The enterprise unites all three departmental clouds into a unified **Hybrid Cloud Architecture**, connecting the on-premises Private Cloud to the Public Cloud via redundant encrypted **AWS Direct Connect / Azure ExpressRoute** pipelines.

Source: Buyya et al., Ch 1, p. 12-18 | Source: Srinivasan & Suresh, Ch 1, p. 15-32

Unit 2: Virtualization, Containers & Cloud Infrastructure (CO302.2)

Topic 2.1: Virtualization Foundations, Significance, Benefits & Taxonomy

GROUP:

Consolidated Questions Covered:

- **Q2:** Explain the concept of Virtualization and describe its significance in modern computing. [BTL2]
- **Q3:** Apply virtualization concepts to analyze its major benefits and applications in cloud infrastructure. Differentiate the major types of virtualization with suitable examples. [BTL3]

1. Academic Definition & Core Concept of Virtualization: **Virtualization** is a foundational technology that creates a software-based (virtual) abstraction of physical computer hardware—including central processing units (CPUs), random-access memory (RAM), storage disks, and network interfaces. It decouples the operating system and software applications from physical hardware constraints, enabling a single physical host server to execute multiple independent, isolated **Virtual Machines (VMs)** concurrently.

2. Significance in Modern Cloud Computing:

- **Enabler of Multi-Tenancy & Resource Pooling:** Virtualization is the core engine that makes cloud computing possible. Without virtualization, cloud providers would have to assign dedicated physical servers to individual customers, destroying economies of scale.
- **Server Consolidation:** Replaces sprawling server rooms where single applications utilized only 10% – 15% of server capacity with dense multi-VM hosting, driving physical hardware utilization above 80%.
- **Rapid Environment Provisioning:** Virtual machines can be instantiated from pre-configured template images in seconds via software APIs, eliminating weeks of hardware procurement.

3. Major Benefits & Applications in Cloud Infrastructure:

1. **Hardware Independence:** VMs are encapsulated as standardized software files; they can be migrated across heterogeneous hardware without re-installing operating systems.
2. **Dynamic Resource Management & Live Migration:** Dynamic Resource Schedulers (e.g., VMware DRS) automatically live-migrate (vMotion) running VMs across host clusters without user downtime to balance CPU/RAM loads.
3. **Fault Isolation & Security Sandboxing:** A catastrophic crash or security breach inside one VM is strictly contained and cannot corrupt other VMs sharing the same physical motherboard.

4. Detailed Taxonomy of Virtualization:

Virtualization Type	Core Concept & Abstraction Layer	Representative Examples
1. Server / Hardware Virtualization	Abstracts physical server CPU, RAM, and motherboard chipset via Hypervisors into complete Virtual Machines.	VMware ESXi, KVM, Microsoft Hyper-V, Xen.
2. Storage Virtualization	Aggregates physical hard drives and SAN arrays across the network into a single consolidated virtual storage pool.	Ceph, VMware vSAN, NetApp SAN-screen.
3. Network Virtualization (SDN)	Decouples network forwarding hardware from control logic, creating software-defined virtual overlay networks and switches.	VMware NSX, Open vSwitch, Cisco ACI, VXLANs.
4. OS-Level Virtualization (Containers)	Virtualizes the operating system kernel, creating multiple isolated user-space instances sharing one host kernel.	Docker, Podman, LXC/LXD, containerd.
5. Desktop Virtualization (VDI)	Hosts desktop operating systems on centralized datacenter servers and streams displays to remote thin clients.	Citrix Virtual Apps, VMware Horizon, Amazon WorkSpaces.

Source: Buyya et al., Ch 3, p. 67-102 | Source: DECAP470, Ch 4, p. 95-115

Topic 2.2: Hypervisors (Type-1 Bare-Metal vs. Type-2 Hosted) & Virtual Machine Architecture**GROUP:****Consolidated Questions Covered:**

- **Q1:** Demonstrate the working principles of Type-I and Type-II Hypervisors with suitable examples. Analyze and compare their architecture, resource utilization, and performance. [BTL3]
- **Q4:** Explain the concept of a Virtual Machine (VM) and describe its major components. [BTL2]
- **Q5:** Illustrate the architecture of a Virtual Machine and explain the role of the hypervisor in managing VMs. [BTL3]

1. Concept of Hypervisor (Virtual Machine Monitor - VMM): A **Hypervisor** or **VMM** is the supervisory software, firmware, or low-level kernel module that creates, runs, and isolates Virtual Machines. It controls physical CPU

instruction execution, arbitrates access to memory page tables, and intercepts privileged hardware calls from guest operating systems.

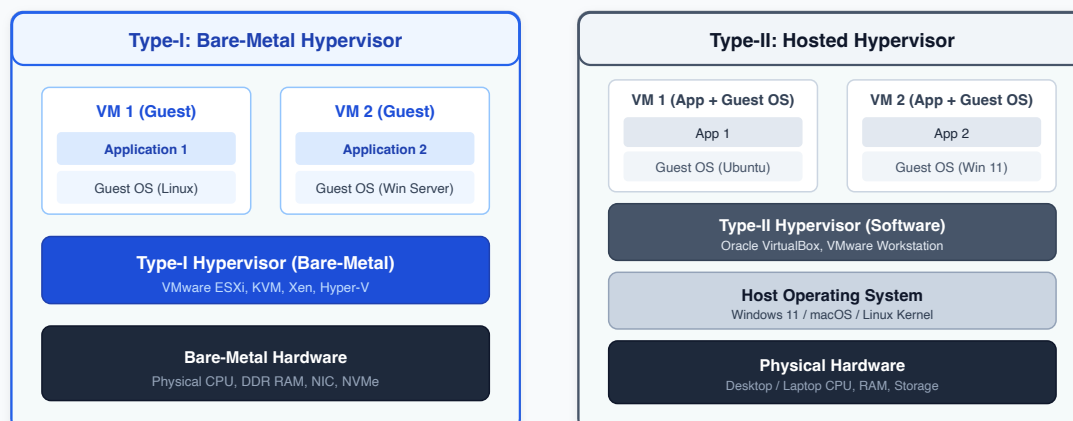


Figure 2.1: Architectural Comparison: Type-I Bare-Metal vs. Type-II Hosted Hypervisor

2. Working Principles & Comparison of Hypervisor Types:

• Type-1 (Bare-Metal / Native) Hypervisor:

- **Working Principle:** Installs directly on the raw physical server hardware (bare metal) without an underlying host operating system. It acts as its own lightweight operating system, managing CPU, memory, and virtual device dispatch directly with sub-millisecond latency.
- **Resource Utilization & Performance:** Extremely high efficiency; near-native hardware execution speed; negligible overhead (2% – 5%).
- **Examples:** VMware ESXi, Microsoft Hyper-V, KVM (Kernel-based Virtual Machine in Linux), Xen, Citrix Hypervisor.

• Type-2 (Hosted) Hypervisor:

- **Working Principle:** Installs and executes as a standard application software program on top of an existing commercial host operating system (e.g., Windows, macOS, Ubuntu Linux). Guest VM hardware calls must pass through the hypervisor, then through the host OS kernel, before reaching physical hardware.
- **Resource Utilization & Performance:** Slower performance due to double OS layer traps; high memory overhead consumed by the host OS.
- **Examples:** Oracle VM VirtualBox, VMware Workstation, VMware Fusion, Parallels Desktop.

3. Parameter-Based Comparison: Type-1 vs. Type-2 Hypervisors:

Parameter	Type-1 (Bare-Metal Hypervisor)	Type-2 (Hosted Hypervisor)
Installation Layer	Runs directly on bare physical server hardware.	Runs as an application inside a Host Operating System.
Host OS Dependency	None; operates independently of any host OS.	Strictly requires an underlying Host OS (Windows, Linux, macOS).
Resource Overhead	Extremely low (2% – 5%); dedicated to guest VMs.	High; Host OS consumes substantial background CPU and RAM.
Performance Speed	Near-native bare-metal execution speed and high I/O throughput.	Lower performance due to dual OS layer context switching.
Security & Isolation	Highest security; minimal attack surface.	Lower security; vulnerability in Host OS compromises all VMs.
Primary Use Cases	Enterprise datacenters, cloud service providers (AWS, Azure).	Software development, classroom labs, local testing.

Parameter	Type-1 (Bare-Metal Hypervisor)	Type-2 (Hosted Hypervisor)
Representative Examples	VMware ESXi, KVM, Microsoft Hyper-V, Xen.	Oracle VirtualBox, VMware Workstation, Parallels.

4. Virtual Machine Architecture & Major Components: A **Virtual Machine (VM)** is a completely isolated software implementation of a physical computer that executes programs like a physical machine:

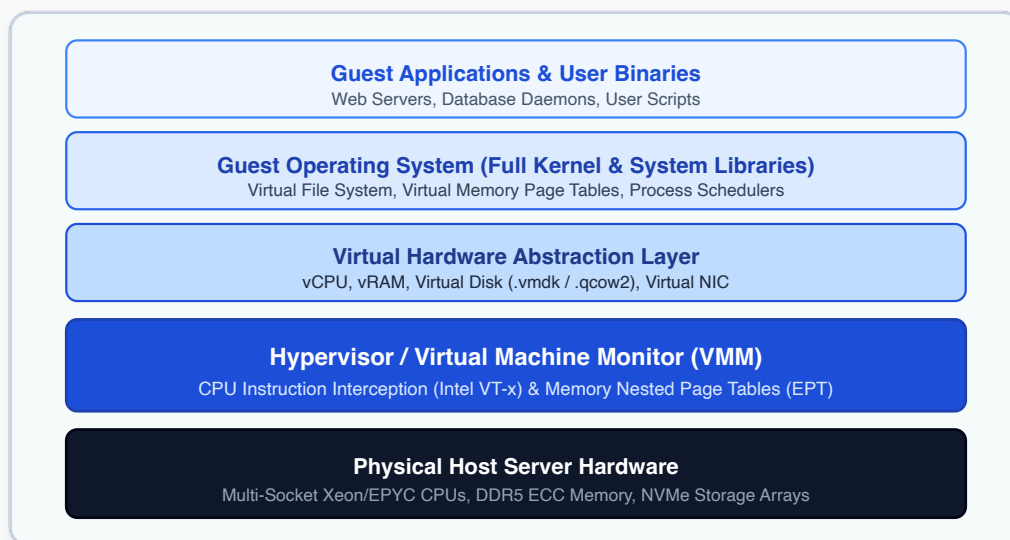


Figure 2.2: Detailed Virtual Machine Architecture and Component Structure

Major Components of a Virtual Machine:

1. **Virtual CPU (vCPU):** Time-sliced allocations of physical CPU execution threads managed by the hypervisor scheduler.
2. **Virtual Memory (vRAM):** Contiguous address space allocated from host physical RAM, mapped via Extended Page Tables (EPT / NPT).
3. **Virtual Disk Image (.vmdk / .qcow2):** Large single files on the host filesystem that represent the entire secondary hard drive of the guest VM.
4. **Virtual Network Interface (vNIC):** Software-emulated network adapters connected to virtual switches with unique virtual MAC addresses.
5. **Guest Operating System:** A standard, unmodified OS (Linux, Windows Server) installed inside the VM.
6. **Virtual BIOS / UEFI Firmware:** Emulated firmware providing boot device selection and hardware initialization during VM power-on.

Source: Buyya et al., Ch 3, p. 75-98 | Source: DECAP470, Ch 4, p. 100-125

Topic 2.3: Virtual Machines vs. Containers & Advantages of Containerization

GROUP:

Consolidated Questions Covered:

- **Q6:** Analyze and compare Virtual Machines and Containers with respect to architecture, resource utilization, isolation, scalability, and application deployment. [BTL2]
- **Q9:** Explain the major advantages of Containers over traditional Virtual Machines. [BTL2]

1. Academic Concept of Containerization: **Containerization** is an Operating System-Level virtualization method that enables running multiple isolated user-space instances (Containers) on a single control host while sharing the same underlying host OS kernel. It leverages two core Linux kernel primitives:

- **Namespaces (Isolation Boundary):** Provides isolated views of system resources (pid for process IDs, net for network interfaces, mnt for filesystem mount points, ipc for inter-process communication).
- **Control Groups / cgroups (Resource Metering):** Restricts and monitors physical hardware consumption (limiting container CPU percentage, RAM allocations, and disk I/O throughput).

2. In-Depth Parameter-Based Comparison Table: Virtual Machines vs. Containers:

Parameter	Virtual Machines (VMs)	Containers (Docker / OCI)
Virtualization Level	Hardware-level virtualization via Hypervisor.	Operating System-level virtualization via Host OS kernel.
Guest OS Requirement	Requires a complete, independent Guest OS per VM (gigabytes of duplicate OS files).	Zero Guest OS; all containers share the single host Linux kernel.
Resource Footprint	Heavy; requires gigabytes of dedicated RAM and storage per instance.	Lightweight; consumes megabytes of RAM; shares read-only base image layers.
Startup Time	Slow (30 seconds to several minutes) due to complete OS boot sequences.	Near-instantaneous (50 milliseconds to 2 seconds); starts as a standard process.
Security Isolation	Extremely high; hardware-enforced isolation via CPU rings; immune to guest crashes.	Moderate / Process-level; container breakout can theoretically compromise shared kernel.
Density per Server	Low density (10 – 50 VMs per physical host server).	High density (100 – 1000+ containers per physical host server).
Portability & Image Size	Heavy monolithic disk images (5 GB – 50 GB); format dependent.	Lightweight image layers (10 MB – 500 MB); runs identically anywhere.
Application Deployment	Suited for monolithic multi-tier enterprise systems and diverse OS kernels.	Ideal for microservice architectures, CI/CD pipelines, and cloud-native apps.

3. Major Advantages of Containers over Virtual Machines:

1. **Sub-Second Auto-Scaling:** Containers spin up in milliseconds, allowing cloud systems to scale instantly during massive real-time web traffic spikes.
2. **Massive Server Density & Cost Savings:** Running hundreds of containers on a single host cuts physical server hardware and cloud VM instance costs by up to 70%.
3. **Immutable Infrastructure & Environment Parity:** Eliminates the classic “**It works on my machine!**” problem by packaging application code, runtime, system tools, and dependencies into immutable image layers that execute identically across developer laptops, test staging, and production clusters.

Source: Buyya et al., Ch 4, p. 90-110 | Source: Handbook of Cloud Computing, Ch 8, p. 210-230

Topic 2.4: Docker Ecosystem Architecture: Images, Containers, Dockerfile, Engine & Registry

GROUP:

Consolidated Questions Covered:

- **Q7:** Explain the fundamentals of Docker and describe the role of Docker images and containers. [BTL2]
- **Q8:** Illustrate the relationship between a Docker image, container, Dockerfile, and Docker Engine. [BTL3]
- **Q12:** Illustrate the basic concepts of Docker and explain the roles of Docker Images, Containers, Dockerfiles, and Docker Registries in application deployment. [BTL3]
- **Q13:** Demonstrate the fundamental concepts of Docker and explain the roles of Docker Images, Containers, Dockerfiles, and Docker Registries in application deployment. [BTL3]

1. Fundamentals of the Docker Ecosystem: Docker is an open-source platform that automates the deployment, scaling, and management of applications inside lightweight, portable software containers. The Docker architecture operates on a Client-Server model coordinated by five core components:

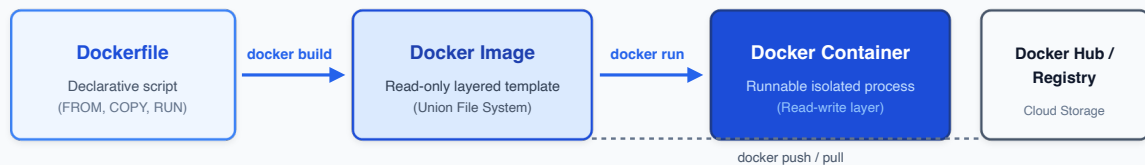


Figure 2.3: Docker Core Workflow Pipeline: Dockerfile to Registry to Container

2. Detailed Roles of Core Docker Components:

1. Dockerfile (Declarative Blueprint):

- A text document containing sequential commands and instructions that a user calls on the command line to assemble an image.
- Key Directives: FROM (base image), WORKDIR (working directory), COPY (transfer files), RUN (execute build commands), ENV (environment variables), EXPOSE (network port documentation), CMD / ENTRYPOINT (default execution process).

2. Docker Image (Immutable Read-Only Template):

- An immutable, read-only package containing application source code, runtime, system libraries, and dependencies.
- Built using a **Union File System (UnionFS)** as a stack of cached read-only layers. Each instruction in a Dockerfile creates a new layer, enabling efficient image sharing and minimal storage duplication.

3. Docker Container (Active Runnable Instance):

- A runnable, isolated instance of a Docker image.
- When a container is instantiated via `docker run`, the Docker Engine adds a thin **Read-Write (R/W) Container Layer** on top of the immutable read-only image layers. All runtime file modifications, logs, and temp files are written to this top layer.

4. Docker Engine / Daemon (dockerd):

- The persistent background server daemon executing on the host OS that builds, runs, networks, and manages Docker containers. It exposes a REST API that the `docker` CLI communicates with.

5. Docker Registry / Docker Hub (Distribution Central):

- A centralized, version-controlled cloud repository for storing, cataloging, and distributing Docker images.
- Public Registries (Docker Hub) host official base images (e.g., python, node, nginx), while Private Registries (AWS ECR, Azure ACR) store proprietary enterprise container images.

3. The End-to-End Build-Ship-Run Lifecycle Relationship:

Dockerfile → (`docker build`) → **Docker Image** → (`docker push/pull`) → **Docker Registry** → (`docker run`) → **Active Container**

Source: Handbook of Cloud Computing, Ch 8, p. 215-235

Topic 2.5: Container Deployment Procedure & Step-by-Step Workflow Using Docker

GROUP:

Consolidated Questions Covered:

- **Q10:** Explain the basic steps involved in container deployment using Docker. [BTL2]
- **Q11:** Demonstrate the basic procedure for deploying an application container using Docker. Identify the major steps involved from creating a Docker image to running the container. [BTL3]

1. Container Lifecycle State Transitions: A Docker container transitions through distinct lifecycle states managed by the Docker daemon:

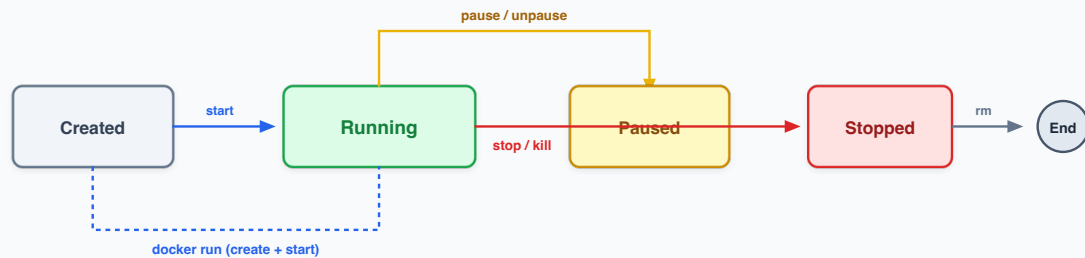


Figure 2.4: Docker Container Lifecycle State Transitions

2. Complete Step-by-Step Procedure for Application Container Deployment:

- **Step 1: Develop Application Code & Dependencies** Create the core application files (e.g., a Python Flask REST web microservice):

```
# app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from SPPU Cloud Computing Docker Container!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

- **Step 2: Construct the Dockerfile Blueprint** Define the container image build steps in a Dockerfile:

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

- **Step 3: Build the Container Image** Execute the build command in the terminal to create the layered image with a version tag:

```
docker build -t ashley/payment-service:v1.0 .
```

- **Step 4: Run & Test Container Locally** Instantiate and run the container in detached background mode (-d) mapping host port 8080 to container port 5000:

```
docker run -d -p 8080:5000 --name payment-app ashley/payment-service:v1.0
# Verify active execution status:
docker ps
```

- **Step 5: Push Image to Registry & Deploy on Production Host** Publish the tested image to Docker Hub and pull it onto the cloud production server:

```
# Authenticate and push to cloud registry:
docker push ashley/payment-service:v1.0

# On Production Cloud Server:
docker pull ashley/payment-service:v1.0
docker run -d --restart=always -p 80:5000 --name prod-payment ashley/payment-service:v1.0
```

3. Essential Docker CLI Management Commands Table:

Docker Command	Technical Function & Description
docker build -t <name> .	Parses Dockerfile and compiles read-only immutable image layers.
docker run -d -p H:C <imgname>	Instantiates and starts a container in background with port forwarding.
docker ps -a	Lists all currently active and stopped container instances with status.
docker logs -f <id>	Streams real-time standard output (stdout/stderr) logs from a container.
docker exec -it <id> /bin/sh	Opens an interactive shell terminal session inside an active running container.
docker stop <id> / docker rm <id>	Sends SIGTERM/SIGKILL to halt a container and removes its container layer.

Source: Handbook of Cloud Computing, Ch 8, p. 225-245

Topic 2.6: Kubernetes Container Orchestration Architecture & Core Components

GROUP:

Consolidated Questions Covered:

- **Q14:** Analyze the role of Kubernetes in container orchestration. Explain its major components and their functions. [BTL2]

1. Definition & Need for Container Orchestration: Kubernetes (K8s) is an open-source container orchestration engine that automates the deployment, horizontal scaling, load balancing, health monitoring, and lifecycle management of containerized application clusters across multi-node host fleets.

- **Why Orchestration is Essential:** While Docker manages individual containers on a single host, enterprise production systems require running thousands of containers across hundreds of virtual machines. Kubernetes eliminates the manual burden of server provisioning, crash recovery, and traffic routing.

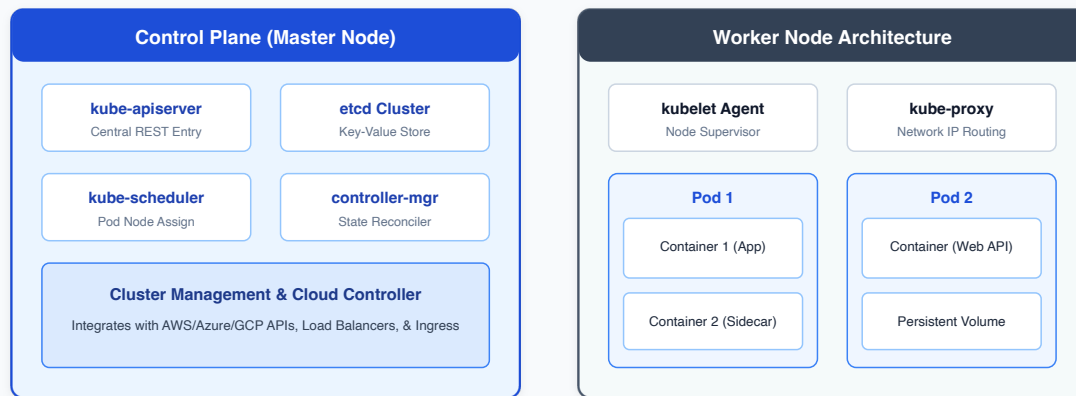


Figure 2.5: Kubernetes Cluster Architecture: Control Plane and Worker Nodes

2. Detailed Breakdown of Kubernetes Cluster Architecture & Components:

• A. Control Plane (Master Node Components) – The Brain of the Cluster:

1. **kube-apiserver:** The central management hub and front-end REST API gateway for the entire cluster. All internal components and external `kubectl` CLI commands communicate exclusively through the API server.
2. **etcd:** A consistent, highly available distributed key-value database that stores the complete cluster state, configuration data, and secret specifications.
3. **kube-scheduler:** Watches for newly created Pods with no assigned node and selects the optimal worker node for them to run on based on hardware resource requirements, affinity rules, and taint constraints.
4. **kube-controller-manager:** Runs background controller loops that regulate cluster state (e.g., Node Controller, Replication Controller, Endpoint Controller), continuously reconciling current state with desired state.

• B. Worker Node Components – The Execution Fleet:

1. **kubelet:** The primary node agent that registers the worker node with the API server, receives Pod specifications (PodSpecs), and instructs the container runtime to launch containers, continuously monitoring their health.
2. **kube-proxy:** A network proxy running on each node that maintains network routing rules, enabling communication to Pods from inside or outside the cluster via IP tables or IPVS.
3. **Container Runtime:** The underlying software responsible for running containers (e.g., `containerd`, `CRI-O`).

3. Key Kubernetes Workload Abstractions:

- **Pod:** The smallest deployable computing unit in Kubernetes, encapsulating one or more tightly coupled containers sharing storage volumes and IP addresses.
- **Deployment:** Declarative controller that manages rolling updates, zero-downtime deployments, and replica counts.
- **Service:** An abstract REST endpoint that provides a permanent, load-balanced virtual IP (ClusterIP, NodePort, LoadBalancer) to access a dynamic group of ephemeral Pods.

Source: Buyya et al., Ch 14, p. 380-410